

Technology Blog Posts by SAP

Making Web Apps IPv6-Ready: The IPv6 Web Resource Checker (webres6)



 philsb1n
Associate

Friday, 2026-05-29



4 Likes



606

SAP MANAGED TAGS

Open Source

 Edit SAP Managed Tags

The transition to IPv6 is well underway. Mobile carriers increasingly run IPv6-only networks, cloud platforms are rolling out IPv6 support by default, and more organizations, e.g. the US federal government, are preparing for a future without IPv4. That naturally raises a practical question for anyone running websites and browser-based SaaS applications:

"Will my application really work in an IPv6-only environment?"

Checking whether a host is reachable via IPv6 is straightforward. What is much harder is understanding whether the *entire* user experience works: HTML, dynamically loaded JavaScript, CSS, fonts, images, analytics, and all the third-party services that modern web apps depend on. To close this gap, we built the **IPv6 Web Resource Checker (webres6)** and released it as open source.

Hosted service: <https://webres6.dev.sap/>

GitHub Project: <https://github.com/SAP/webres6>

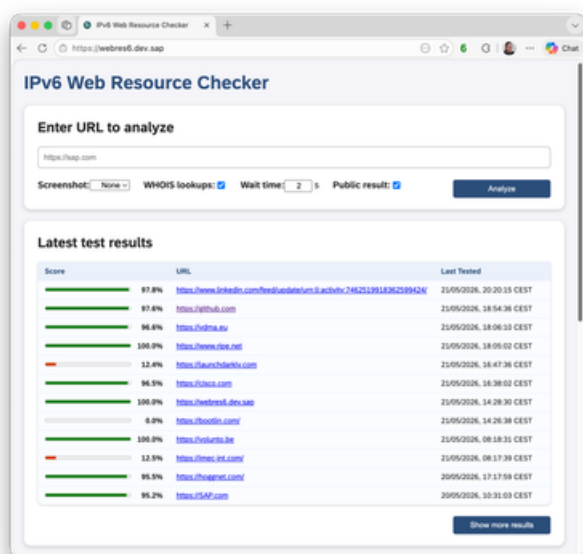
License: Apache 2.0

From Prototype to Open Source

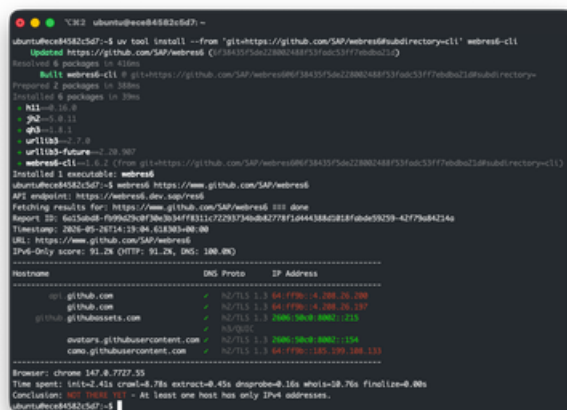
Everything started as an internal prototype roughly a year ago... We built a command-line tool to drive a browser via Selenium, collected the network performance log, and produced a textual summary of which resources loaded over IPv6 and which did not. It worked technically, but the results were hard to share with other teams or integrate into broader team processes. Especially requesting IPv6 at the client side turned out to be problematic as not all offices had been deployed with IPv6 connectivity yet.

We therefore pivoted to an API backend checking IPv6-only readiness and a web frontend rendering a human-readable report. This approach also gave us an opportunity to dog-food the tool itself. SAP Gardener, our open source Kubernetes Orchestrator, had just introduced [Dual-stack and IPv6-only cluster support](#), and we wanted to put it to the test: We

decided to deploy the hosted service on an IPv6-only Kubernetes cluster on AWS with a dual-stack ingress and NAT64 capable egress.



As we envisioned the tool was not



only of interest for SAP developers, we decided to share it with the broader developer and IPv6 community by releasing it as open-source and presented it at the [IETF 124 Hackathon](#). Community feedback then shaped further development: users asked for **permanent, shareable result URLs**, DNS analysis and a scoreboard of recent public results similar to [ssllabs.com](#). We also added OpenTelemetry support for

metrics and traces, result caching via Valkey, S3 archiving, and an experimental MCP server that has proven surprisingly capable.

Today, the [IPv6 Resource Checker](#) runs as a hosted service in an IPv6-only Gardener-based Kubernetes environment. For self-hosting deployments, we have public Docker images available. The CLI part is still available, but now uses the service to perform the analysis and is easily [installable also using uv](#). A simple idea – pivoting from a local CLI to a SaaS tool with shareable reports – made the tool far more useful for real-world usage by our development teams and proved how SaaS and open-source flourish together.

How webres6 Works

At its core, webres6 answers one question: *what exactly does a browser load when it visits this URL, and over which IP protocol are those resources reachable?*



To do that, it launches a real browser in a container via Selenium WebDriver. The browser renders the target page as a user would – executing JavaScript, and retrieving deferred assets as the application logic demands. While the page loads, webres6 extracts every network request from the Chromium performance logs and summarizes the results by host the data is fetched from.

Each distinct host is then classified as IPv6-capable or not, based on the protocol used to fetch the resources and calculate an IPv6-Only HTTP score as the share of assets that have been fetched using IPv6. In addition, the host names are resolved via an IPv6-only DNS recursor to examine the DNS delegation chain and check whether the authoritative

path to each host is reachable from an IPv6-only-strict resolver. Based on the share of assets that would be prevented from loading by IPv6-only-strict DNS, we calculate a separate IPv6-Only DNS score. The compiled report highlights exactly which parts of the page are available over IPv6, which still depend on IPv4, and provides hints which third party may be in charge to fix the IPv6 gaps.

What We Learned by Testing Real Applications

The main entry point is easy to get IPv6-ready – the surrounding ecosystem is not. In most environments we tested after the initial IPv6-enablement, the primary application host just worked: modern load balancers, CDNs, and reverse proxies handle IPv6 well. However, once you look at the full set of resources the browser loads, the picture changes. Modern web applications depend on a wide ecosystem of external providers including third party analytics tools, error tracking agents, ... In an IPv6-only network, they can cause blank icons, missing charts, incomplete UI layouts, and subtle JavaScript errors that are entirely invisible in a dual-stack environment. Because these integrations are owned by different teams – marketing, security, operations – they can easily fall outside the scope of any IPv6 project. The positive side of this is that webres6 provides concrete, shareable evidence that makes it straightforward to approach providers with specific findings rather than abstract concerns.

DNS deserves special attention. During the whole project, we occasionally encountered hard debug issues that turned out to be DNS related. First, we noticed that AWS's DNS resolvers didn't work with IPv6-only DNS zones. While this is a strange corner case we only encountered on test sites so far, it came to a surprise and made us rely on Google DNS instead. Second, we figured out that Chromium and Google Chrome do only implement Happy Eyeballs v1 so far. As our deployment of webres6 is IPv6-only with NAT64, they can't fall back to IPv4 in cases where AAAA DNS records exist but IPv6 connectivity is broken. Finally, we found a corner case with CDNs where not all DNS servers support IPv6. This can lead to certain DNS servers giving up resolving the host names. These issues are now known by the vendors and fixes are underway.

Running SaaS service in IPv6-only landscapes is feasible. In retrospect, running webres6 as hosted service in a Gardener managed IPv6-only Kubernetes cluster worked out well and with much less friction than expected. While dog-fooding on this allowed us to fix several smaller issues in Gardener and identify some IPv6-only limitations in AWS recursive DNS and Google Chrome, we now have the confidence to build selected new components on IPv6-only infrastructure.

Get Started

If you want to try webres6, the simplest path is the hosted service at <https://webres6.dev.sap/>. Start with a few representative URLs – your main public site, an important self-service portal – and review which hosts appear as IPv4-only and what they are used for – only keep in mind that the tester is limited to applications that don't require a login. Share the report URL with your infrastructure or third-party provider contacts and move from a general "we should think about IPv6" conversation to concrete, actionable steps.

We hope webres6 is useful on your IPv6 journey. If you test it against your own applications, we would be glad to hear what you find – in the comments below or as issues and pull requests on [GitHub](#).

Author: Philipp S. Tiesel, Director Internet Standards, SAP Research & Innovation

LABELS

Technology Updates

TAGS

IPv6

Open Standards

Testing

